# Danh sách GitHub repo hỗ trợ tiết kiệm token cho LLM

**Cập nhật:** 28-10-2026

**Phạm vi:** 41 repo công khai (43 dòng khi tách ba biến thể LLMLingua) cho nén prompt/context, coding agent, cache, memory, retrieval, routing, serving và đo usage.

**Với prompt tiếng Việt:** đây là các công cụ LLM dùng chung, không phải benchmark riêng cho tiếng Việt. Hãy kiểm tra tỷ lệ token và chất lượng trả lời trên chính bộ prompt tiếng Việt của bạn.

## Cách đọc phần trăm

- Ưu tiên số đo thực nghiệm của bài báo, tài liệu dự án và benchmark độc lập; ghi rõ benchmark, metric, cỡ mẫu và điểm hạn chế. Số tự báo cáo được đánh dấu riêng.
- Với mức nén **N×**, phần token đầu vào giảm là **1 - 1/N**. Ví dụ **20×** tương ứng tối đa **95%** ít token đầu vào hơn.
- **Input token**, **output token**, **byte context**, **token biểu diễn trong index**, **chi phí** và **độ trễ** là các metric khác nhau; không cộng, xếp hạng hay thay thế cho nhau.
- **N/A** nghĩa là sau khi rà README, benchmark và tài liệu/paper liên quan, chưa tìm thấy tỷ lệ giảm thực nghiệm gắn trực tiếp với repo. Đây không phải số 0. Mức giảm chi phí do chọn model rẻ hơn không được gọi là giảm token.
- Kết quả khác mô hình, tokenizer, tác vụ, baseline và cỡ mẫu; benchmark của tác giả không tương đương xác nhận độc lập. Không có repo nào được thử trên bộ prompt tiếng Việt trong các nguồn dưới đây.

## 1. Nén prompt và context

Đây là nhóm có tác động trực tiếp và đo được nhất lên **input tokens**. Có thể dùng một biến thể riêng hoặc tích hợp vào pipeline RAG.

| Repo / biến thể | Phương pháp | % token giảm được | Căn cứ và giới hạn |
|---|---|---|---|
| microsoft/LLMLingua — bản gốc | Xếp hạng và loại token ít quan trọng trong prompt | **95.1% input token trong một ca GSM8K** | Bảng 2 của paper EMNLP: full-shot **2,366** token, EM **78.85**; LLMLingua quarter-shot **117** token, EM **77.33** (giảm **1.52** điểm). Đây là so sánh benchmark với số shot khác nhau, không phải bảo đảm giữ nguyên chất lượng trong mọi prompt. |
| microsoft/LLMLingua — LongLLMLingua | Nén context dài có xét câu hỏi, ưu tiên đoạn liên quan | **50–83% input token ở mức nén 2×–6×** | Paper ACL: prompt khoảng 10k token; NaturalQuestions có ca GPT-3.5 Turbo đạt gần **4×** ít token hơn và tăng hiệu suất tối đa **21.4%**; LooGLE báo cáo giảm chi phí **94%**. Các số là theo task/model, không khái quát sang mọi truy vấn. |
| microsoft/LLMLingua — LLMLingua-2 | Nén trích xuất dựa trên phân loại token | **50–80% input token ở mức nén 2×–5×** | Paper ACL Findings: đánh giá trên MeetingBank, LongBench, ZeroScrolls, GSM8K và BBH; báo cáo tốc độ end-to-end **1.6×–2.9×**. Tỷ lệ nén là cấu hình thử, không phải mức token giảm đồng đều trên mọi tác vụ. |

## 2. Memory cho hội thoại dài

| Repo | Phương pháp | Tỷ lệ theo nguồn | Căn cứ và giới hạn |
|---|---|---|---|
| mem0ai/mem0 | Lưu thông tin thành memory rồi truy xuất theo câu hỏi thay vì gửi toàn bộ lịch sử | **>72% ít token context hơn, ước tính từ benchmark hiện hành** | Tài liệu đánh giá ghi dưới **7,000** token/retrieval ở ngân sách top-200; full-context trên cùng nhóm benchmark thường từ **25,000** token trở lên, nên cận dưới suy ra là **1 − 7,000/25,000 >72%**. Paper LOCOMO 2025 còn báo cáo **>90%** token cost; đó là metric/thiết lập cũ, không nên nhập làm một. |
| getzep/graphiti | Knowledge graph theo thời gian, truy xuất sự kiện và quan hệ phù hợp | **53% ít context trong benchmark sản phẩm Zep** | Benchmark Zep đo median context **5,760 → 2,680** token trên LoCoMo; độ chính xác đi cùng giảm **94.7% → 86.5%** (**−8.2** điểm). Đây là kết quả sản phẩm Zep có Graphiti làm nền cùng thành phần proprietary, không phải đo trực tiếp repo Graphiti OSS. |

## 3. Retrieval, RAG và lọc tài liệu

Các repo này giúp lấy phần dữ liệu phù hợp thay vì chèn toàn bộ kho tài liệu vào prompt. Mức tiết kiệm do cấu hình retrieval, số chunk, kích thước chunk và độ dài tài liệu quyết định.

| Repo | Phương pháp | % token giảm được | Căn cứ và giới hạn |
|---|---|---|---|
| run-llama/llama_index | Index, retrieval, reranking và tối ưu context RAG | **N/A — cần benchmark** | Framework hỗ trợ lọc context; không có tỷ lệ chung cho mọi index và **top-k**. |
| lancedb/lancedb | Vector/multimodal database cho retrieval | **N/A — cần benchmark** | Có thể chỉ lấy các đoạn liên quan; cần đo số token sau khi render context. |
| stanford-futuredata/ColBERT | Neural retrieval; nén biểu diễn token của tài liệu trong index | **65–70% ít biểu diễn token trong index** | Paper pruning trên model kiểu ColBERT giữ hiệu năng khi index chỉ còn khoảng **30–35%** token biểu diễn. Đây là RAM/disk của index, không phải prompt token gửi cho LLM và không phải phép đo cài repo chính thức. |
| neuml/txtai | Semantic search và pipeline retrieval | **N/A — cần benchmark** | Repo không cam kết một phần trăm tiết kiệm token tổng quát. |
| Unstructured-IO/unstructured | Trích xuất, chuẩn hóa và chia nhỏ tài liệu | **N/A — cần benchmark** | Có thể lọc phần dư thừa, nhưng không bảo đảm giảm token; kết quả phụ thuộc tài liệu, metadata và cấu hình chunking. |
| sinaptik-ai/pandas-ai | Tạo truy vấn trên dữ liệu có cấu trúc thay vì gửi cả bảng | **N/A — cần benchmark** | Có thể tránh đưa cả bảng vào context; thực tế phụ thuộc schema, câu hỏi và SQL/Python được tạo. |

## 4. Semantic cache

| Repo | Phương pháp | % token giảm được | Cách tính đúng |
|---|---|---|---|
| zilliztech/GPTCache | Lưu câu trả lời; truy vấn tương tự có thể được trả từ cache | **Chưa có % token tổng phiên đáng tin cậy; mỗi hit tránh request LLM đó** | Ví dụ synthetic trong repo có **904** cache positive đúng, **92** positive sai và **3** miss trên **999** câu hỏi; đây là hit count, không phải token-weighted saving và có false hit. Tính % thực tế theo tổng token của các request được cache, trừ chi phí embedding/lookup. |

## 5. Routing và gateway

Nhóm này chủ yếu tối ưu **chi phí hoặc lựa chọn model**. Chọn model rẻ hơn không tự động làm số token ít đi; input/output token có thể thay đổi theo model và chất lượng câu trả lời.

| Repo | Phương pháp | % token giảm được | Tác động chính |
|---|---|---|---|
| BerriAI/litellm | Gateway, fallback, định tuyến theo độ khó và cache | **Không thấy % token giảm chung; cost giảm 45% trên SWE-bench** | A/B 25 task: capability router giải **23/25** task như Opus 5, nhưng cost **$11.15** so với **$20.27** (**−45%**); một lần thử/task, tập nhỏ. Production case báo cost giảm **51.1%** trên **272,876** request nhưng dùng giá lại cùng token về model đắt hơn, không phải A/B model response. Đây là cost, không phải token-count. |
| Portkey-AI/gateway | Gateway đa provider, policy, cache và quản lý request | **Không thấy % token giảm; báo cáo cost giảm trung bình 38% nhờ cache** | LLMs in Prod ’25 tổng hợp hơn **2T** token ẩn danh, **650+** nhóm và **90+** khu vực trong năm 2024. Đây là phân tích traffic nền tảng do Portkey công bố, không phải A/B có đối chứng; cost không đồng nghĩa token. |
| aurelio-labs/semantic-router | Phân loại ý định để chọn model/workflow | **N/A — không có mức chung** | Có thể tránh dùng model lớn cho request đơn giản; phần giảm chủ yếu là chi phí. |

## 6. Quan sát và đánh giá

Các repo dưới đây giúp tìm prompt dài, request lặp hoặc model đắt; bản thân công cụ quan sát/đánh giá không cắt token cho đến khi có người sửa pipeline.

| Repo | Vai trò | % token giảm trực tiếp | Ghi chú |
|---|---|---|---|
| langfuse/langfuse | Trace, đánh giá và theo dõi token/chi phí | **Không tự giảm token** | Hỗ trợ tìm chỗ lãng phí để tối ưu sau đó. |
| comet-ml/opik | Tracing và đánh giá ứng dụng LLM | **Không tự giảm token** | Không thay đổi prompt hoặc output nếu chưa cấu hình tối ưu. |
| Helicone/helicone | Theo dõi request, usage và chi phí | **Không tự giảm token** | Công cụ đo lường; không có phần trăm tiết kiệm mặc định. |
| Arize-ai/phoenix | Quan sát trace, retrieval và chất lượng | **Không tự giảm token** | Hỗ trợ chẩn đoán RAG; token giảm phụ thuộc thay đổi sau đánh giá. |
| openlit/openlit | OpenTelemetry cho usage, độ trễ và chi phí LLM | **Không tự giảm token** | Telemetry không tự rút gọn context. |
| microsoft/promptflow | Xây, chạy và đánh giá flow LLM | **N/A — cần benchmark** | Flow có thể được tối ưu; kết quả phụ thuộc thiết kế. |
| promptfoo/promptfoo | So sánh prompt/model và kiểm tra chất lượng | **Không tự giảm token** | Hỗ trợ chọn cấu hình đạt chất lượng với ít chi phí hơn. |
| stanford-crfm/helm | Benchmark model theo nhiều tiêu chí | **Không tự giảm token** | Hỗ trợ chọn model; không trực tiếp giảm token. |
| optexai/token-bench | Chạy A/B benchmark cho RTK, Headroom, CodeGraph và Context Mode | **Không tự giảm token** | Đây là harness đo input, cache-read, output, chi phí và pass rate. Phần tái phân tích CSV công khai cho Context Mode ở mục đối chứng bên dưới là phép tính của báo cáo này, không phải kết luận của repo benchmark. |

## 7. Tối ưu prompt, output và cấu trúc

| Repo | Phương pháp | % token giảm được | Căn cứ và giới hạn |
|---|---|---|---|
| stanfordnlp/dspy | Tối ưu chương trình/prompt theo metric và chọn model phù hợp | **Chưa có % token chung; tài liệu ghi một ca Shopify cost rẻ hơn khoảng 75×** | Ví dụ GEPA: tối ưu prompt và chuyển tác vụ GPT-5 sang Qwen nhỏ; đo cost/task và độ tin cậy, không phải 75× ít token. |
| dottxt-ai/outlines | Ràng buộc output theo schema/grammar | **N/A — cần benchmark** | Có thể giảm output sai định dạng và lần gọi lại; không có mức giảm chung. |
| 567-labs/instructor | Output có cấu trúc và xác thực schema | **N/A — cần benchmark** | Số lần retry có thể giảm; đo cả token của lần thử lỗi và validation. |
| microsoft/semantic-kernel | Orchestration, plugin và quản lý memory | **N/A — cần benchmark** | Framework không tự động giảm context; tùy cách triển khai. |

## 8. Local inference và serving

Nhóm này có thể giảm chi phí hạ tầng hoặc tăng thông lượng. Với cùng prompt và cách sinh, nó **không trực tiếp làm giảm số token**.

| Repo | Phương pháp | % token giảm trực tiếp | Lợi ích chính |
|---|---|---|---|
| ggml-org/llama.cpp | Chạy model local/quantized | **Không tự giảm token** | Có thể giảm chi phí API/hạ tầng; token phụ thuộc prompt và model. |
| vllm-project/vllm | Serving throughput cao và quản lý bộ nhớ | **Không tự giảm token** | Tăng hiệu suất GPU/requests, không tự rút prompt. |
| sgl-project/sglang | Runtime serving và tối ưu thực thi | **Không tự giảm token** | Tối ưu latency/throughput, không phải token compression. |
| lm-sys/FastChat | Serving, huấn luyện và đánh giá model mở | **Không tự giảm token** | Tự host có thể giảm chi phí API; không có phần trăm token giảm trực tiếp. |

## 9. Coding agent: giảm output, tool context và việc làm thừa

Nhóm này gần với cách Ponytail hoạt động: hướng agent viết đúng phần cần thiết, chọn context gọn hơn hoặc nén kết quả tool. Mỗi số bên dưới đo một đại lượng khác nhau; không cộng các tỷ lệ với nhau.

| Repo | Cách hỗ trợ | Metric giảm được | Căn cứ và giới hạn |
|---|---|---|---|
| DietrichGebert/ponytail | Skill YAGNI cho coding agent: đọc luồng trước, tái dùng thứ sẵn có, tránh code và lời giải thừa | **Tự báo cáo: −45% output token, −26% cost. Độc lập: −7.5% tổng token (không ý nghĩa thống kê)** | Benchmark dự án: Claude Code + Opus 5.5, 39 task, 5 lượt/arm; output token **−45%** (CI 95% **−54%…−35%**), cost **−26%**. JetBrains, 80 cặp task: tổng token **−7.5%** (**p=.12**, không có ý nghĩa), output **−13.8%** (**p=.001**), cost **−10.3%** (**p=.004**), không thấy khác biệt chất lượng. |
| rtk-ai/rtk | Hook viết lại lệnh shell để lọc log, gom kết quả và bỏ output lặp | **Claim 60–90% shell output; thử A/B độc lập không giảm token toàn phiên** | JetBrains, 80 cặp task sạch: input mới **+3.2%** (**p=.23**), cost **+7.6%** (**p=.004**) ở effort thấp; cost **+0.1%** (**p=.99**) ở effort cao. Hook chỉ chạm khoảng **20%** ký tự tool-result; trần lý thuyết gần **3%** input token. Tỷ lệ shell output không phải tiết kiệm cả phiên. |
| chopratejas/headroom | Proxy wire-level nén request trước khi gửi provider | **Pilot độc lập: 10.2% token bỏ khỏi request được chạm; tổng input phiên +43%** | tokbench, **N=1** mỗi arm: proxy xác nhận bỏ **342K** token (**10.2%** trung bình trên phần request được chạm), nhưng provider vẫn tính nhiều hơn arm native **43%** vì phiên có **237** thay vì **180** lượt. Pilot nhỏ, không đại diện chung. |

| Repo | Cách hỗ trợ | Metric giảm được | Căn cứ và giới hạn |
|---|---|---|---|
| yvgude/lean-ctx | Quản lý context bằng MCP, lưu và truy xuất nội dung theo nhu cầu | **Pilot độc lập: input token toàn phiên +38% ở cấu hình MCP** | tokbench, **N=1** mỗi arm, báo cáo provider-billed input **2.28M** native so với **3.14M** lean-ctx MCP; dashboard sản phẩm báo **0** token tiết kiệm trong run đó. Một task/harness; không suy ra kết quả chung. |
| sliday/tamp | Proxy nén tool output và lịch sử gửi lặp; nhiều mức từ lossless đến lossy | **47.56% token ở mức L5; 47.61% ở L9** | Whitepaper v0.8: 12 tình huống × 18 cấu hình = 216 lượt live A/B qua OpenRouter; bộ chấm độc lập xác nhận 216/216 lượt giữ task completion. Từ L5 lên L9 chỉ thêm **0.05** điểm phần trăm; không nên xem mức nén cực đại là cải thiện tương xứng. |
| mksglu/context-mode | Xử lý log, test, CSV và snapshot ngoài context; lưu dữ liệu để truy vấn phần cần thiết | **Bytes: −96%; A/B dự án: cost −10–64%; dữ liệu công khai: token −25.7% theo tái phân tích** | Benchmark bytes: 21 tình huống, **376 KB → 16.5 KB**, không đánh giá task cuối. A/B dự án: 17 kết quả ghép cặp, Claude Code, list-price cost thấp hơn **10–64%**, khoảng tin cậy 95% nằm dưới 0 và task success bằng nhau; đây không phải hóa đơn API thực. Tái phân tích CSV token-bench: 10 nhóm task/model, **262** trial có Context Mode và **314** baseline; trung bình đồng trọng số theo nhóm giảm **25.7%** tổng token và **22.6%** cost ước tính, pass **88.8%** so với **88.6%**. Bộ dữ liệu cộng đồng không ngẫu nhiên/ghép cặp; kết quả này là phép tính của báo cáo. |
| Supercompress/Supercompress | Nén context theo câu hỏi, giữ nguyên các đoạn bằng chứng liên quan | **64.1% token context trong B5** | README/benchmark: 24 ca coding agent, **16,647 → 5,148** token, 24/24 lượt giữ evidence; mức cao nhất **96.6%** ở retention ≥99%. Đây là evidence-containment test, chưa đo downstream LLM task success; trung bình toàn bộ 390 ca chỉ **3.6%**. |
| microsoft/TokenEconomizer | Học micro-skill theo repo, bỏ tool ít dùng và thoát vòng lặp lỗi | **18.4–30.9% cost trung bình; không phải token-count** | Paper SWE-Bench: **277** task, 6 repo, 5 cấu hình agent–model; cả 30 ô repo×setting đều giảm cost và đã cộng cả model call của plugin. Pass@1 tăng 4/5 setting; riêng Opus giảm **1.0** điểm pass@1. |
| JuliusBrussee/caveman | Hướng assistant bỏ câu đệm nhưng giữ nguyên chi tiết kỹ thuật | **A/B độc lập: output token −8.5%, không có ý nghĩa thống kê** | JetBrains: 82 task ghép cặp, 3 lượt, Claude Sonnet 5 low; **592k → 542k** output token (**−8.5%**, **p=.82**), không thấy khác biệt chất lượng. Chi phí tổng bị một outlier kéo lên; Tura `N=2` ghi tổng token **−4.03%**, nhưng mẫu nhỏ và độ dao động lớn. |
| Aider-AI/aider | Tạo repo map symbol quan trọng, lấy phần liên quan theo trạng thái chat | **N/A — chưa thấy % giảm thực nghiệm chung** | Tài liệu repo map đặt ngân sách mặc định 1,000 token và điều chỉnh động; ngân sách map không phải mức giảm so với full context. |
| yamadashy/repomix | Chọn file vào prompt; **--compress** trích chữ ký class/hàm bằng Tree-sitter | **Thử nghiệm độc lập: −3.5% đến −40.5% tổng repo; có loại file tăng +30.1%** | Đo trên Nuxt: toàn site **−40.5%**, TypeScript **−78.7%**, Vue component **+30.1%**. Tái lập trên psf/requests, median 3 lượt: compress-only **−19.0%**; giữ corpus có SVG lớn chỉ **−3.5%**. Kết quả phụ thuộc ngôn ngữ, file và cấu hình include. |

## Đối chứng A/B và thử nghiệm độc lập

Mỗi hàng so sánh trong chính benchmark của nó. Không xếp hạng trực tiếp giữa các hàng: model, tác vụ, cách tính token và số lần chạy khác nhau.

| Repo và nguồn | Kết quả đo được | Giới hạn cần đọc cùng con số |
|---|---|---|
| Ponytail — JetBrains | 80 cặp task, 251 lượt tính phí: tổng token **−7.5%** (**p=.12**, không ý nghĩa), output **−13.8%** (**p=.001**), cost **−10.3%** (**p=.004**). | Claude Code + Sonnet 5 medium; chất lượng không khác có ý nghĩa. Đây là kiểm chứng độc lập nhỏ hơn mức **−45% output** tự báo cáo của dự án. |
| RTK — JetBrains | 86 task, 80 cặp sạch: effort thấp cost **+7.6%** (**p=.004**), input mới **+3.2%** (**p=.23**); effort cao cost **+0.1%** (**p=.99**). | Claude Code + Sonnet 5; chỉ khoảng 20% ký tự tool-result nằm trong phần hook có thể chạm. Riêng benchmark output 12 lệnh ước lượng **87–88%** giảm shell output, không phải token toàn phiên. |

| Repo và nguồn | Kết quả đo được | Giới hạn cần đọc cùng con số |
|---|---|---|
| Caveman — JetBrains | 82 task ghép cặp: output token **592k → 542k** (**−8.5%**, **p=.82**). | Không có ý nghĩa thống kê về mức giảm; chất lượng không đổi đáng kể. Cost tổng bị một outlier làm tăng; không lấy số đó làm tỷ lệ tiết kiệm chung. |
| Ponytail, RTK, Caveman — Tura | Một task chuyển repo Rust sang Python, 2 lượt/arm: tổng token lần lượt **−7.56%**, **+13.20%**, **−4.03%**. | **N=2**, GPT-5.6 Sol High; biên độ dao động token của Ponytail **57.36%**, cost **51.69%**. Chỉ nên xem là tín hiệu thăm dò, không phải kết quả ổn định. |
| Headroom và lean-ctx — tokbench | Pilot **N=1**: Headroom bỏ được **342K** token khỏi request được nén (**10.2%**), nhưng provider-billed input toàn run **+43%**; lean-ctx MCP **+38%** input. | Nhiều lượt hơn giải thích vì sao giảm ở payload không đảm bảo giảm tổng run: Headroom **237** so với **180** lượt. Tất cả quality gates trong task pilot vẫn pass. |
| Context Mode — tái phân tích CSV token-bench | 10 nhóm task/model: tổng token trung bình đồng trọng số theo nhóm **−25.7%**, cost ước tính **−22.6%**, pass **88.8%** so với **88.6%**. | Phép tính của báo cáo này trên dữ liệu công khai: 262 trial có Context Mode, 314 baseline; các run không ngẫu nhiên/ghép cặp, gồm hai phiên bản và ngày chạy khác nhau. Đây là đối chứng cộng đồng sơ bộ. |
| Repomix — đo thực nghiệm | Repo **psf/requests**, 3 lần lấy median: **−3.5%** khi giữ mọi file; **−19.0%** khi bỏ thư mục asset lớn. | Cùng corpus nhưng tính năng kiểm tra bảo mật mặc định còn bỏ 3 file (gồm 2 file test); khi tắt kiểm tra, số token tăng **34.5%**. Nên xác nhận file được đưa vào trước khi so phần trăm. |

## Gợi ý chọn nhanh

- **Muốn giảm input token ngay:** thử LLMLingua-2 hoặc LongLLMLingua trên tập prompt đại diện; bắt đầu ở mức nén vừa rồi kiểm tra độ chính xác.
- **Agent có lịch sử dài:** so sánh Mem0 với full-context trên hội thoại của bạn; tính cả token dùng để trích xuất/cập nhật memory.
- **Request hay lặp lại:** thử GPTCache; báo cáo tỷ lệ hit theo token và kiểm tra freshness/false hit.
- **RAG tài liệu:** bắt đầu với retrieval (ví dụ LlamaIndex + LanceDB/ColBERT), rồi mới thêm compressor nếu retrieval vẫn đưa quá nhiều context.
- **Chỉ muốn hóa đơn thấp hơn:** gateway/routing hoặc local serving có thể phù hợp, nhưng theo dõi riêng **input_tokens**, **output_tokens**, **cost** và chất lượng.
- **Dùng coding agent:** nếu cần ít code/output thừa, thử Ponytail; nếu muốn giảm output shell, thử RTK/Tamp; nếu agent đọc nhiều log, thử Context Mode/Headroom. Đo lại cả tổng token, cost, số lượt và task pass.
- **Đừng xếp hạng % khác loại:** ví dụ **45%** output token, **64%** token context, **96%** byte context và **26%** cost đo các đại lượng khác nhau.

## Nguồn chính và cách áp dụng

- LLMLingua EMNLP paper, LongLLMLingua ACL paper, LLMLingua-2 ACL Findings.
- Mem0 benchmark docs, Graphiti nền tảng Zep benchmark, ColBERT token pruning paper.
- LiteLLM 25-task A/B, LiteLLM production cost accounting, Portkey LLMs in Prod ’25.
- Ponytail benchmark dự án, Tamp whitepaper v0.8, Context Mode A/B docs, Supercompress README/benchmark, TokenEconomizer paper.
- JetBrains independent Ponytail A/B, RTK A/B, Caveman A/B, Tura small-sample A/B, tokbench pilot.
- token-bench public data, Repomix corpus experiment, Repomix file-type test.

**Kết luận:** có số đo cụ thể cho một số repo, nhưng rất ít số là “giảm X% token của cả phiên” trong thử nghiệm độc lập đủ lớn. Khi hai nguồn mâu thuẫn, ưu tiên kết quả A/B độc lập và nêu rõ nếu chỉ có benchmark của tác giả, pilot nhỏ hoặc mức giảm nằm ở output/context con. Với repo còn **N/A**, nguồn công khai đã rà không có tỷ lệ thực nghiệm phù hợp; không tự bịa phần trăm để lấp bảng.